Guía de Implementación — Template RBAC

Referencia personal para arrancar nuevos proyectos basados en este template. Cubre setup completo, arquitectura de seguridad y los pasos concretos para adaptar el sistema a las reglas de negocio de cada proyecto.

Índice

1. Qué ofrece este template
2. Setup inicial de un nuevo proyecto
3. Arquitectura de seguridad
4. Adaptar el RBAC a un nuevo dominio
5. Agregar un nuevo recurso — Backend
6. Agregar un nuevo recurso — Frontend
7. Variables de entorno y checklist de producción
8. Referencia rápida

1. Qué ofrece este template

Capa	Tecnología	Propósito
Backend	FastAPI + SQLModel	API REST con auth JWT
Base de datos	PostgreSQL 17	Persistencia vía SQLAlchemy ORM
Migraciones	Alembic	Control de versiones del schema

Capa	Tecnología	Propósito
Frontend	Next.js 15 + TypeScript	SPA con gestión de auth y permisos
Estilos	Tailwind CSS v4	UI con soporte dark/light mode
Infraestructura	Docker Compose	Levanta PostgreSQL en local

Sistema de permisos: `recurso:acción` — granular, heredado por roles, con bypass para superusuarios.

Flujo de autenticación: JWT stateless con access token (30 min) + refresh token (7 días). El frontend renueva el access token automáticamente ante un 401.

Auditoría: Toda operación CRUD queda registrada con usuario, acción, recurso y dirección IP.

---

2. Setup inicial de un nuevo proyecto

2.1 Clonar y renombrar

```bash
# Clonar el template
git clone <repo> mi-proyecto
cd mi-proyecto

# Iniciar repo limpio (opcional)
rm -rf .git
git init
git add .
git commit -m "chore: init from rbac template"
```

2.2 Configurar variables de entorno

```bash
```

```
cp backend/.env.example backend/.env
```

Editar `backend/.env` con los valores del nuevo proyecto:

```env
PROJECT_NAME=Mi Proyecto
VERSION=1.0.0

# Generar con: python -c "import secrets;
print(secrets.token_hex(32))"
SECRET_KEY=<clave-aleatoria-de-64-chars>

ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7

POSTGRES_SERVER=localhost
POSTGRES_USER=mi_proyecto_user
POSTGRES_PASSWORD=<password-seguro>
POSTGRES_DB=mi_proyecto_db
POSTGRES_PORT=5432

BACKEND_CORS_ORIGINS=["http://localhost:3000"]
```

Crear `frontend/.env.local`:

```env
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
```

2.3 Levantar la base de datos

```bash
# Ajustar credenciales en docker-compose.yml para que coincidan
con .env
docker compose up -d

# Verificar que PostgreSQL está healthy
docker compose ps
```

2.4 Backend

```bash
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Crear tablas e inicializar datos semilla
python -c "from app.db.init_db import create_tables, init_db;
create_tables(); init_db()"

# Levantar servidor
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

API disponible en: `http://localhost:8000/docs`

2.5 Frontend

```bash
cd frontend
npm install
npm run dev
```

App disponible en: `http://localhost:3000`

2.6 Usuarios de prueba por defecto

Usuario	Contraseña	Rol	Superuser
superadmin	admin123	Super Admin	Sí
admin	admin123	Admin	No
manager	manager123	Manager	No
user	user123	User	No

> **Cambiar estas contraseñas antes de cualquier deploy.**

---

3. Arquitectura de seguridad

3.1 Diagrama de flujo de autenticación

```
[Login]
  │
  ▼
POST /api/v1/auth/login (form-urlencoded: username + password)
  │
  ├── Backend valida con bcrypt
  ├── Genera access_token (HS256, 30 min, claim type="access")
  └── Genera refresh_token (HS256, 7 días, claim type="refresh")
        │
        ▼
  Frontend guarda ambos en localStorage
  Access token se envía en cada request: Authorization: Bearer
<token>

[Request protegido]
  │
  ▼
  HTTPBearer extrae el token del header
  verify_token() decodifica y valida claim type="access"
  get_current_user() busca el usuario en DB por sub (username)
  get_current_active_user() verifica is_active=True
  require_permissions() verifica que el usuario tenga los permisos
requeridos
        │
        ├── is_superuser=True → bypass, acceso concedido
        └── Compara recurso:acción contra permisos heredados de
roles activos
```

```
[Token expirado (401)]
  │
  ▼
  Frontend intercepta el 401
  POST /api/v1/auth/refresh (body JSON: refresh_token)
        │
        ├── Refresh válido → nuevo access_token + refresh_token →
reintenta request
        └── Refresh inválido → dispara evento "auth:expired" →
logout automático
```

3.2 Cómo funciona `require_permissions()` — Backend

Archivo: `backend/app/core/deps.py`

```python
def require_permissions(required_permissions: List[str]):
    def permission_checker(current_user: User =
Depends(get_current_active_user)):
        if current_user.is_superuser:
            return current_user  # Superusers bypasan todo

        user_permissions = [
            f"{permission.resource}:{permission.action}"
            for role in current_user.roles
            if role.is_active                          # Solo
roles activos
            for permission in role.permissions
            if permission.is_active                    # Solo
permisos activos
        ]

        for required in required_permissions:
            if required not in user_permissions:
                raise HTTPException(status_code=403,
detail=f"Permission denied. Required: {required}")

        return current_user
    return permission_checker
```

Uso en un endpoint:

```python
@router.get("/productos")
def listar_productos(
    current_user: User =
Depends(require_permissions(["productos:read"]))
):
    ...
```

Helpers predefinidos (agregar los propios en `deps.py`):

```python
def require_user_read():
    return require_permissions(["users:read"])
```

3.3 Tokens JWT — detalle

Ambos tokens usan el claim `type` para evitar que un refresh token se use como access token y viceversa.

```python
# security.py

# Access token — expira en 30 min
to_encode["type"] = "access"
jwt.encode(to_encode, SECRET_KEY, algorithm="HS256")

# Refresh token — expira en 7 días
to_encode["type"] = "refresh"
jwt.encode(to_encode, SECRET_KEY, algorithm="HS256")
```

`verify_token()` rechaza cualquier token con `type != "access"`.
`verify_refresh_token()` rechaza cualquier token con `type != "refresh"`.

3.4 Manejo de refresh concurrente — Frontend

Archivo: `frontend/lib/api/client.ts`

Si múltiples requests reciben 401 simultáneamente, el cliente serializa el refresh:

```typescript
// Solo un intento de refresh a la vez; el resto espera la misma promesa
if (this.isRefreshing && this.refreshPromise) {
    return this.refreshPromise;
}
```

Una vez obtenido el nuevo token, todos los requests reintentados lo usan.

3.5 Modelo de permisos

```
Usuario → (N roles) → (N permisos)

Permiso = { resource: "productos", action: "read" }
          → name = "productos:read"
```

Acciones estándar: `create`, `read`, `update`, `delete`, `export`

Los permisos se heredan de **todos** los roles activos del usuario, sin jerarquía entre roles. Si un permiso está en cualquier rol activo, el usuario lo tiene.

3.6 Registro de auditoría

Todo endpoint que modifica datos llama a `audit_service.log()`:

```python
from app.services.audit_service import audit_service

audit_service.log(
    db=db,
    user_id=current_user.id,
    username=current_user.username,
    action="create",          # create | update | delete | login |
```

```
logout
    resource="productos",
    resource_id=str(nuevo.id),
    details={"nombre": nuevo.nombre},
    ip_address=request.client.host,
)
```

Los logs son consultables vía `GET /api/v1/audit/logs` (requiere `audit:read`).

---

4. Adaptar el RBAC a un nuevo dominio

4.1 Definir los recursos del proyecto

Antes de tocar código, mapear los recursos y acciones propios del dominio:

```
# Ejemplo: sistema de gestión de inventario
recursos:
  - productos:   create, read, update, delete, export
  - categorias:  create, read, update, delete
  - proveedores: create, read, update, delete
  - inventario:  read, update, export
  - reportes:    read, export

# Más los recursos base del template (siempre necesarios):
  - users, roles, permissions: create, read, update, delete
  - dashboard: read
  - audit: read
  - settings: read, update
```

4.2 Editar `init_db.py`

Archivo: `backend/app/db/init_db.py`

Reemplazar la lista `permissions_data` con los permisos del nuevo dominio:

```python
permissions_data = [
    # Permisos base (mantener siempre)
    {"name": "users:create", "resource": "users", "action": "create", "description": "Crear usuarios"},
    {"name": "users:read",   "resource": "users", "action": "read",   "description": "Ver usuarios"},
    # ... resto de users, roles, permissions, dashboard, audit, settings

    # Permisos del dominio específico
    {"name": "productos:create", "resource": "productos", "action": "create", "description": "Crear productos"},
    {"name": "productos:read",   "resource": "productos", "action": "read",   "description": "Ver productos"},
    {"name": "productos:update", "resource": "productos", "action": "update", "description": "Editar productos"},
    {"name": "productos:delete", "resource": "productos", "action": "delete", "description": "Eliminar productos"},
    {"name": "productos:export", "resource": "productos", "action": "export", "description": "Exportar productos"},
    # ...
]
```

4.3 Definir roles y asignar permisos

En la misma función `init_db()`, ajustar los roles y sus permisos:

```python
roles_data = [
    {"name": "Super Admin",    "description": "Acceso total al sistema"},
    {"name": "Administrador",  "description": "Administración completa excepto eliminación de permisos"},
    {"name": "Supervisor",     "description": "Gestión operativa"},
    {"name": "Operador",       "description": "Carga y consulta de datos"},
    {"name": "Consultor",      "description": "Solo lectura"},
]
```

```
# Asignación de permisos por rol (después de crear los roles)
admin = next(r for r in roles if r.name == "Administrador")
admin.permissions = [p for p in permissions if p.name not in
["permissions:delete"]]

supervisor = next(r for r in roles if r.name == "Supervisor")
supervisor.permissions = [p for p in permissions
    if p.action in ["read", "update"]
    or (p.resource == "productos" and p.action == "create")]
```

4.4 Agregar helpers de permisos en `deps.py`

```python
# backend/app/core/deps.py

def require_producto_read():
    return require_permissions(["productos:read"])

def require_producto_create():
    return require_permissions(["productos:create"])

def require_producto_update():
    return require_permissions(["productos:update"])

def require_producto_delete():
    return require_permissions(["productos:delete"])
```

---

5. Agregar un nuevo recurso protegido

Checklist completo para agregar, por ejemplo, un recurso `Producto`.

5.1 Modelo ORM

`backend/app/models/models.py`

```python
class Producto(SQLModel, table=True):
    __tablename__ = "productos"

    id: Optional[int] = Field(default=None, primary_key=True)
    nombre: str = Field(index=True)
    descripcion: Optional[str] = None
    precio: float
    is_active: bool = Field(default=True)
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: Optional[datetime] = None
```

5.2 Schemas Pydantic

`backend/app/schemas/schemas.py`

```python
class ProductoCreate(BaseModel):
    nombre: str
    descripcion: Optional[str] = None
    precio: float

class ProductoUpdate(BaseModel):
    nombre: Optional[str] = None
    descripcion: Optional[str] = None
    precio: Optional[float] = None
    is_active: Optional[bool] = None

class ProductoRead(BaseModel):
    id: int
    nombre: str
    descripcion: Optional[str]
    precio: float
    is_active: bool
    created_at: datetime
```

5.3 Servicio CRUD

`backend/app/services/crud.py`

```python
class ProductoService:
    def get(self, db: Session, id: int) -> Optional[Producto]:
        return db.get(Producto, id)

    def get_all(self, db: Session, skip: int = 0, limit: int =
100):
        return
db.exec(select(Producto).offset(skip).limit(limit)).all()

    def create(self, db: Session, data: ProductoCreate) ->
Producto:
        producto = Producto(**data.model_dump())
        db.add(producto)
        db.commit()
        db.refresh(producto)
        return producto

    def update(self, db: Session, id: int, data: ProductoUpdate) -
> Optional[Producto]:
        producto = self.get(db, id)
        if not producto:
            return None
        for key, value in
data.model_dump(exclude_unset=True).items():
            setattr(producto, key, value)
        producto.updated_at = datetime.utcnow()
        db.commit()
        db.refresh(producto)
        return producto

    def delete(self, db: Session, id: int) -> bool:
        producto = self.get(db, id)
        if not producto:
            return False
        db.delete(producto)
        db.commit()
        return True

producto_service = ProductoService()
```

5.4 Endpoint con permisos y auditoría

`backend/app/api/productos.py`

```python
from fastapi import APIRouter, Depends, HTTPException, Request
from sqlmodel import Session
from app.db.database import get_db
from app.core.deps import require_permissions
from app.models.models import User
from app.services.crud import producto_service
from app.services.audit_service import audit_service
from app.schemas.schemas import ProductoCreate, ProductoRead,
ProductoUpdate

router = APIRouter(prefix="/productos", tags=["productos"])

@router.get("/", response_model=list[ProductoRead])
def listar_productos(
    db: Session = Depends(get_db),
    current_user: User =
Depends(require_permissions(["productos:read"])),
):
    return producto_service.get_all(db)

@router.post("/", response_model=ProductoRead, status_code=201)
def crear_producto(
    data: ProductoCreate,
    request: Request,
    db: Session = Depends(get_db),
    current_user: User =
Depends(require_permissions(["productos:create"])),
):
    producto = producto_service.create(db, data)
    audit_service.log(
        db=db,
        user_id=current_user.id,
        username=current_user.username,
        action="create",
        resource="productos",
        resource_id=str(producto.id),
```

```
        details={"nombre": producto.nombre},
        ip_address=request.client.host,
    )
    return producto

@router.put("/{producto_id}", response_model=ProductoRead)
def actualizar_producto(
    producto_id: int,
    data: ProductoUpdate,
    request: Request,
    db: Session = Depends(get_db),
    current_user: User =
Depends(require_permissions(["productos:update"])),
):
    producto = producto_service.update(db, producto_id, data)
    if not producto:
        raise HTTPException(status_code=404, detail="Producto no
encontrado")
    audit_service.log(db=db, user_id=current_user.id,
username=current_user.username,
                      action="update", resource="productos",
resource_id=str(producto_id),
                      details=data.model_dump(exclude_unset=True),
ip_address=request.client.host)
    return producto

@router.delete("/{producto_id}", status_code=204)
def eliminar_producto(
    producto_id: int,
    request: Request,
    db: Session = Depends(get_db),
    current_user: User =
Depends(require_permissions(["productos:delete"])),
):
    if not producto_service.delete(db, producto_id):
        raise HTTPException(status_code=404, detail="Producto no
encontrado")
    audit_service.log(db=db, user_id=current_user.id,
username=current_user.username,
                      action="delete", resource="productos",
```

```
resource_id=str(producto_id),
                        details={}, ip_address=request.client.host)
```

5.5 Registrar el router

`backend/app/main.py`

```python
from app.api import productos

app.include_router(productos.router, prefix=settings.API_V1_STR)
```

5.6 Actualizar la lista de recursos disponibles

El endpoint `/permissions/resources/available` tiene los recursos **hardcodeados**. Al agregar un recurso nuevo, actualizarlo en `backend/app/api/permissions.py`:

```python
@router.get("/resources/available")
def get_available_resources(current_user: User =
Depends(require_permission_read())):
    return {
        "resources": [
            "users", "roles", "permissions",
            "dashboard", "reports", "settings",
            "productos",   # <-- agregar el nuevo recurso aquí
        ]
    }
```

Sin este paso, la UI de administración de permisos no mostrará el nuevo recurso en el selector.

5.7 Migración Alembic

```bash
```

```
cd backend
alembic revision --autogenerate -m "add productos table"
alembic upgrade head
```

---

6. Nueva página en el frontend

Checklist completo para agregar la página `/productos` al frontend.

6.1 Tipos TypeScript

`frontend/types/index.ts`

```typescript
export interface Producto {
  id: number;
  nombre: string;
  descripcion?: string;
  precio: number;
  is_active: boolean;
  created_at: string;
  updated_at?: string;
}

export interface CreateProductoDTO {
  nombre: string;
  descripcion?: string;
  precio: number;
}

export interface UpdateProductoDTO {
  nombre?: string;
  descripcion?: string;
  precio?: number;
  is_active?: boolean;
}

export interface GetProductosParams {
```

```
  page?: number;
  size?: number;
  search?: string;
  is_active?: boolean;
}
```

6.2 Servicio API

`frontend/lib/api/services.ts`

```typescript
export const productoService = {
  getAll: (params?: GetProductosParams) =>
    apiClient.get<PaginatedResponse<Producto>>(`/productos?
${buildQuery(params)}`),

  getById: (id: number) =>
    apiClient.get<Producto>(`/productos/${id}`),

  create: (data: CreateProductoDTO) =>
    apiClient.post<Producto>('/productos', data),

  update: (id: number, data: UpdateProductoDTO) =>
    apiClient.put<Producto>(`/productos/${id}`, data),

  delete: (id: number) =>
    apiClient.delete<void>(`/productos/${id}`),
};
```

6.3 Crear la página

Crear el archivo `frontend/app/productos/page.tsx`. El patrón estándar combina la guarda de auth, la carga de datos y los controles protegidos por permiso:

```tsx
'use client';
import { useEffect, useState, useCallback } from 'react';
import { useRouter } from 'next/navigation';
```

```
import { useAuth } from '@/context/AuthContext';
import { useToast } from '@/context/ToastContext';
import DashboardLayout from '@/components/layout/DashboardLayout';
import ProtectedComponent from
'@/components/common/ProtectedComponent';
import { productoService } from '@/lib/api/services';
import type { Producto } from '@/types';

export default function ProductosPage() {
  const { isAuthenticated, isLoading, hasPermission } = useAuth();
  const { showToast } = useToast();
  const router = useRouter();

  const [productos, setProductos] = useState<Producto[]>([]);
  const [loading, setLoading] = useState(true);

  // Guarda de autenticación y permiso mínimo
  useEffect(() => {
    if (!isLoading && !isAuthenticated) router.push('/login');
    if (!isLoading && isAuthenticated &&
!hasPermission('productos:read')) router.push('/dashboard');
  }, [isAuthenticated, isLoading, hasPermission, router]);

  const fetchProductos = useCallback(async () => {
    try {
      setLoading(true);
      const data = await productoService.getAll({ page: 1, size:
20 });
      setProductos(data.items);
    } catch {
      showToast('Error al cargar productos', 'error');
    } finally {
      setLoading(false);
    }
  }, [showToast]);

  useEffect(() => {
    if (isAuthenticated) fetchProductos();
  }, [isAuthenticated, fetchProductos]);

  if (isLoading || loading) return <div>Cargando…</div>;
```

```
  return (
    <DashboardLayout>
      <div className="space-y-4">
        <div className="flex justify-between items-center">
          <h1 className="text-xl font-semibold">Productos</h1>

          {/* Solo visible si tiene permiso de crear */}
          <ProtectedComponent permissions={['productos:create']}>
            <button onClick={() => { /* abrir modal */ }}>
              Nuevo producto
            </button>
          </ProtectedComponent>
        </div>

        {/* Tabla de datos */}
        {productos.map(p => (
          <div key={p.id}>
            <span>{p.nombre}</span>

            {/* Acciones protegidas individualmente */}
            <ProtectedComponent permissions=
{['productos:update']}>
              <button onClick={() => { /* editar */
}}>Editar</button>
            </ProtectedComponent>
            <ProtectedComponent permissions=
{['productos:delete']}>
              <button onClick={() => { /* eliminar */
}}>Eliminar</button>
            </ProtectedComponent>
          </div>
        ))}
      </div>
    </DashboardLayout>
  );
}
```

6.4 Agregar al sidebar de navegación

`frontend/components/layout/Sidebar.tsx`

Agregar una entrada al array `navItems` con el ícono, la ruta y el permiso requerido para verla. El sidebar oculta automáticamente los ítems que el usuario no puede ver.

```tsx
import { Package } from 'lucide-react'; // elegir ícono de lucide-
react

const navItems: NavItem[] = [
  { label: 'Dashboard',   href: '/dashboard',   icon:
LayoutDashboard, permissions: ['dashboard:read'] },
  { label: 'Mi Perfil',   href: '/profile',     icon: User },
  { label: 'Usuarios',    href: '/users',        icon: Users,
permissions: ['users:read'] },
  { label: 'Productos',   href: '/productos',    icon: Package,
permissions: ['productos:read'] }, // ← nueva entrada
  { label: 'Roles',       href: '/roles',        icon: Shield,
permissions: ['roles:read'] },
  { label: 'Permisos',    href: '/permissions',  icon: Key,
permissions: ['permissions:read'] },
  { label: 'Auditoría',   href: '/audit',        icon:
ClipboardList,   permissions: ['audit:read'] },
];
```

Los ítems sin `permissions` siempre se muestran (como “Mi Perfil”). El sidebar ya maneja el filtrado con `ProtectedComponent` internamente.

6.5 Proteger secciones de UI con `ProtectedComponent`

```tsx
import ProtectedComponent from
'@/components/common/ProtectedComponent';

// Botón visible solo con un permiso específico
<ProtectedComponent permissions={['productos:create']}>
  <Button onClick={handleCreate}>Nuevo Producto</Button>
</ProtectedComponent>

// Visible si tiene ALGUNO de los permisos
<ProtectedComponent permissions={['productos:update',
```

```
'productos:delete']}>
  <RowActions />
</ProtectedComponent>

// Visible solo si tiene TODOS los permisos
<ProtectedComponent permissions={['reportes:read',
'reportes:export']} requireAll>
  <ExportButton />
</ProtectedComponent>
```

6.6 Verificar permisos directamente en código

Usar `useAuth` cuando la lógica es más compleja que mostrar/ocultar un elemento:

```tsx
import { useAuth } from '@/context/AuthContext';

function FilaProducto({ producto }: { producto: Producto }) {
  const { hasPermission, hasAnyPermission } = useAuth();

  const puedeEditar  = hasPermission('productos:update');
  const puedeEliminar = hasPermission('productos:delete');
  const puedeActuar  = hasAnyPermission(['productos:update',
'productos:delete']);

  return (
    <tr>
      <td>{producto.nombre}</td>
      {puedeActuar && (
        <td>
          {puedeEditar  && <button>Editar</button>}
          {puedeEliminar && <button>Eliminar</button>}
        </td>
      )}
    </tr>
  );
}
```

---

7. Variables de entorno y checklist de producción

Variables obligatorias en producción

```env
# Backend
SECRET_KEY=<mínimo 64 chars aleatorios, nunca el default>
ACCESS_TOKEN_EXPIRE_MINUTES=30     # ajustar según sensibilidad
del sistema
REFRESH_TOKEN_EXPIRE_DAYS=7        # ajustar según política de
sesiones

POSTGRES_SERVER=<host-produccion>
POSTGRES_USER=<usuario-especifico-del-proyecto>
POSTGRES_PASSWORD=<password-fuerte>
POSTGRES_DB=<db-especifica-del-proyecto>

BACKEND_CORS_ORIGINS=["https://mi-dominio.com"]

# Frontend
NEXT_PUBLIC_API_URL=https://api.mi-dominio.com/api/v1
```

Checklist antes de deploy

- [ ] `SECRET_KEY` generada con `secrets.token_hex(32)` (no el valor del `.env.example`)
- [ ] Contraseñas de usuarios semilla cambiadas o usuarios semilla eliminados
- [ ] `BACKEND_CORS_ORIGINS` restringido al dominio real (no `localhost`)
- [ ] PostgreSQL no expuesto públicamente (acceso solo desde la red interna)
- [ ] HTTPS configurado (nginx/caddy como reverse proxy)
- [ ] Variables de entorno en el servidor, nunca en el repositorio
- [ ] `DEBUG=False` / modo producción en FastAPI si se agrega esa variable
- [ ] Revisar que `_debug` files no estén en producción (`.gitignore` o eliminar)

Generar SECRET_KEY

```bash
python -c "import secrets; print(secrets.token_hex(32))"
```

---

8. Referencia rápida

Estructura de archivos clave

```
backend/
├── app/
|   ├── core/
|   |   ├── config.py        # Settings (Pydantic), variables de
entorno
|   |   ├── security.py      # JWT, bcrypt — no modificar salvo
agregar algoritmos
|   |   └── deps.py          # Dependencias de FastAPI:
get_current_user, require_permissions
|   ├── models/
|   |   └── models.py        # Tablas SQLModel: User, Role,
Permission, AuditLog
|   ├── schemas/
|   |   └── schemas.py       # DTOs Pydantic de entrada/salida
|   ├── services/
|   |   ├── crud.py          # Lógica CRUD: UserService,
RoleService, PermissionService
|   |   └── audit_service.py # Registro de auditoría
|   ├── api/
|   |   ├── auth.py          # /auth/login, /auth/refresh,
/auth/logout
|   |   ├── users.py         # /users/ CRUD + roles
|   |   ├── roles.py         # /roles/ CRUD + permisos
|   |   ├── permissions.py   # /permissions/ CRUD
|   |   └── audit.py         # /audit/logs
|   └── db/
|       ├── database.py      # Engine SQLAlchemy, get_db()
|       └── init_db.py       # Datos semilla: permisos, roles,
```

```
usuarios
└── alembic/
    └── versions/            # Historial de migraciones

frontend/
├── context/
|   ├── AuthContext.tsx      # Estado global de auth: user, token,
login, logout, hasPermission
|   ├── ThemeContext.tsx     # Dark/light mode
|   └── ToastContext.tsx     # Notificaciones
├── lib/
|   └── api/
|       ├── client.ts        # ApiClient: fetch + refresh
automático + manejo de 401
|       └── services.ts      # Servicios por recurso: userService,
roleService, etc.
├── components/
|   └── common/
|       └── ProtectedComponent.tsx  # Muestra/oculta UI según
permisos
├── types/
|   └── index.ts             # Interfaces TypeScript: User, Role,
Permission, etc.
└── app/                     # Páginas Next.js App Router
```

Comandos frecuentes

```bash
# Levantar DB
docker compose up -d

# Backend
cd backend && source venv/bin/activate
uvicorn app.main:app --reload --port 8000

# Re-inicializar datos semilla (idempotente)
python -c "from app.db.init_db import init_db; init_db()"

# Nueva migración
alembic revision --autogenerate -m "descripcion"
```

```
alembic upgrade head
alembic downgrade -1

# Frontend
cd frontend && npm run dev

# Logs de Docker
docker compose logs -f postgres
```

Endpoints de autenticación

Método	Endpoint	Body	Descripción
POST	`/api/v1/auth/login`	`username`, `password` (form-data)	Login, retorna access + refresh token
POST	`/api/v1/auth/refresh`	`{"refresh_token": "..."}`	Renueva access token
POST	`/api/v1/auth/logout`	— (requiere Bearer)	Logout (auditado)

Checklist completo para un nuevo recurso

Backend:

```
1. models/models.py          → Clase SQLModel con table=True
2. schemas/schemas.py        → DTOs Create / Read / Update
3. services/crud.py          → Clase XService con
get/get_all/create/update/delete + singleton
4. api/X.py                  → Router con endpoints y
require_permissions(["X:action"])
5. main.py                   → app.include_router(X.router,
prefix=settings.API_V1_STR)
6. api/permissions.py        → Agregar "X" a la lista de
resources/available
7. db/init_db.py             → Permisos
```

```
X:create/read/update/delete y asignarlos a roles
8. core/deps.py              → Helpers opcionales
require_X_read(), require_X_create(), etc.
9. alembic                   → alembic revision --autogenerate -m
"add X table"
                               alembic upgrade head
```

Frontend:

```
1. types/index.ts            → Interfaces X, CreateXDTO,
UpdateXDTO, GetXParams
2. lib/api/services.ts       → xService con
getAll/getById/create/update/delete
3. app/X/page.tsx            → Página con guarda de auth +
fetchData + DashboardLayout
4. components/layout/Sidebar.tsx → Entrada en navItems con
permissions: ["X:read"]
5. En la página              → ProtectedComponent para botones de
crear/editar/eliminar
```